

ARE 212 SECTION 12: Power Calculations

Fiona Burlig

April 14, 2016

Before you embark on a research project, it's useful to know whether your data will actually be able to answer your research question. The first step is having the right data - but the second (often overlooked) step is having *enough* of that data. This week, we'll talk about power calculations, which are the savvy econometrician's way of figuring this out.¹ We'll start with a discussion of what statistical power is, and then work through some simulations using both fake and real data.²

Off we go

Loosely, you'll be "powered" to answer a research question if there's enough variation in your data to answer that question. The easiest way of thinking about research questions is to think about comparing a "treatment" group and a "control" group.³ As long as you can do this, you can do a power calculation. We'll think through power calculations using the "gold standard" of empirical research designs: the randomized controlled trial (RCT).

The idea is as follows: suppose you're interested in testing whether fertilizer use increases farmers' yields. You'd like to run the following regression:

$$\text{yield}_f = \alpha + \beta \text{fertilizer}_f + \varepsilon_f$$

where $\text{fertilizer}_f = 1$ if farmer f is using fertilizer. Since you're a brilliant ARE 212 student, you're very concerned about omitted variable bias/endogeneity, because farmers who use fertilizer already might be better crop managers, have better fields, etc. All of these things will lead to higher yields, be correlated with fertilizer use, and also be correlated with your regression's error term. Rather than trying to find a clever IV (since we know how hard that is), you decide to run your own experiment.

You get permission from the Kenyan government to run an experiment in economics' RCT Mecca, Busia. You'll gather a sample of farmers, and then flip a coin (or, more realistically, run `rbinom()` in R) to determine whether each farmer in your sample is going to be in the treatment group or the control group. Farmers in your treatment group receive a free bag of fertilizer. (Suppose for the purposes of our toy example that everybody who gets a bag

¹Less savvy econometricians dive headlong into projects, only to find out six months later that there's not enough variation in their data to actually estimate anything useful. Don't be one of those.

²This section was inspired by Meredith Fowlie's excellent lecture notes, my [ongoing work](#) with Louis Preonas and Matt Woerman, and a useful afternoon with Erin Kelley and Deirdre Sutula. Thank (blame?) them next time you see them.

³Though there are a variety of research designs out there (many of which you'll see in Michael Anderson's class next semester), in many of the designs applied econometricians use, it's easy to imagine splitting your data up into these two groups. It's a little harder to make this distinction with structural estimation - but in "reduced form" work, the randomized controlled trial; instrumental variables; regression discontinuity design (which is almost always just a fancy form of IV anyway); matching; difference-in-differences; and panel fixed effects models (to name a few) can all be thought of this way. Slight modifications of these techniques will also work with non-binary treatment, but we won't get into that today.

of fertilizer uses the whole bag, does not resell their fertilizer, and otherwise behaves “well” in the experimental sense.)⁴ Farmers in your control group do not receive fertilizer. You come back at the end of the harvest season and collect data on yields from all farmers. Why is this awesome? By virtue of random assignment into treatment, treatment is uncorrelated with your error term. In other words, you have, on average, identical farmers in your treatment and control group. Any differences in yields between them in your endline survey must be a result of the treatment itself!⁵ If you run the following regression:

$$\text{yield}_f = \alpha + \beta \text{treatment}_f + \varepsilon_f$$

you’ll get a consistent estimate of $\hat{\beta}$. Excellent.

Before you can run this experiment, you need to figure out how many farmers you’re going to include in your study. You’ll want to include just enough farmers to make sure that you get a statistically significant treatment effect. Your experiment is going to be expensive, so you want to make sure that you’re neither substantially over-powered, nor significantly under-powered - you want a Goldilocks sample size: juuuuuust right.

How wrong can you be?

A test’s “size” and “power” are all about making mistakes. In hypothesis testing land, a Type I error occurs when we reject a true null hypothesis. We usually do hypothesis testing with a size of $\alpha = 0.05$ - this is where 95% confidence intervals come from - which means that we’re choosing to accept the fact that we’ll incorrectly reject the null 5 times out of 100. In our case, the null hypothesis will (almost always) be that our treatment effect is equal to zero.

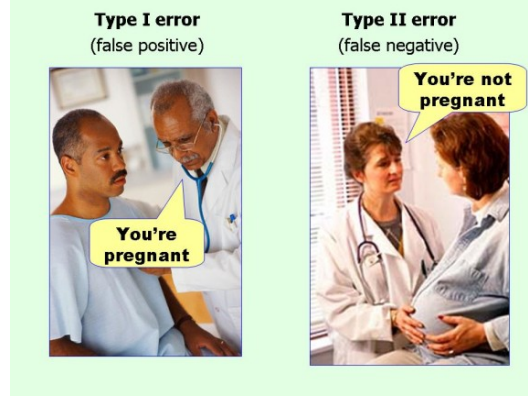
On the other hand, a Type II error occurs when we fail to reject a false null hypothesis. Thinking about this requires an alternative hypothesis. In our case, the alternative hypothesis will be that our treatment effect is equal to (the true population) β . When we design experiments, we try to “power” them to $\kappa = 0.80$, meaning that 80% of experiments that we conduct with a given sample size will correctly reject the null when the null is false. Notice that $95\% > 80\%$: as a discipline, we’ve (arbitrarily?) decided that we’re more concerned about Type I errors than we are about Type II errors.

Still confused about the distinction between Type I and Type II errors? This illustration might help:⁶

⁴We can deal with non-compliance with an approach of converting our intent-to-treat estimate into an average treatment effect like we discussed last week, but we’ll gloss over that for now.

⁵This is a widely used method - it’s basically the only accepted way of doing medical drug testing in the US today.

⁶Shamelessly stolen from Marc Bellemare’s [blog](#).



A few important notes: An increase in (the absolute value of) β will increase power, all else equal. The further the alternate hypothesis is from the null, the less likely we are to fail to reject the null when the null is false. Also, the stricter we are about our Type I error rate, the lower the power we'll have: the only way that we can guarantee that we'll never falsely reject a true null is to reject *all* nulls - this means that we'll also end up failing to reject false nulls. Finally, the larger your sample, the smaller the size of your test and the larger its power. More N helps everything.

Work / Time

It turns out that, given a desired level of power κ and a size α , we'll reject the null if:

$$\beta > (t_{1-\kappa} + t_\alpha) \sqrt{\mathbb{V}(\hat{\beta})}$$

in other words, we'll reject the null if the true effect is greater than the standard error of our estimate, scaled by a test statistic which captures both our power and our size.⁷ We call the smallest τ for which this holds the minimum detectable effect (MDE).

In a regular OLS RCT setup, we can substitute in for the variance of $\hat{\beta}$ and write:

$$MDE = (t_{1-\kappa} + t_\alpha) \sqrt{\frac{1}{P(1-P)}} \sqrt{\frac{\sigma^2}{N}}$$

where N is the sample size, P is the fraction of units in the treated group, and σ^2 is our residual variance.⁸ We can rearrange this equation to solve for N :

$$N = \frac{(t_{1-\kappa} + t_\alpha)^2}{P(1-P)} \frac{\sigma^2}{MDE^2}$$

The good news is that we know $t_{1-\kappa}$, t_α , and P . We have to pick the MDE - there's no standard in the literature; we have to guess based on previous work and our best idea about the magnitude of the treatment effect. Notice that once we've got our data, we can easily estimate σ^2 : note that this is the *residual* variance, so the best way to estimate this thing is to save the residuals from your regression and calculate their variance. But we don't know what the variance in our data looks like until we've run our experiment! You can either substitute in your best

⁷ t_α is the familiar 1.96 when $\alpha = 0.05$, and t_κ is 0.58 when $\kappa = 0.8$.

⁸Set $P = 0.5$ to ensure the smallest necessary sample size. If you ever set $P < 0.5$ in the real world, it'll be due to your (binding) budget constraint.

guess of σ^2 , based on the existing literature, or you can take existing data that looks similar to the data your experiment will generate to make a good guess of this important parameter.

Once you know $\kappa, \alpha, P, \sigma^2$, and have chosen our desired MDE, we can easily calculate the necessary sample size. Let's do this in R. As always, we'll start with our set-up.

```
library(parallel)
library(dplyr)
library(ggplot2)

##### FUNCTIONS
as.tbl_df <- function(data) {
  dataset <- as.data.frame(data) %>%
    tbl_df()
}

tbl_dfGrabber <- function(data, varnames) {
  matrixObject <- data %>%
    select_(.dots = varnames) %>%
    as.matrix()
}

OLS <- function(data, y, X) {
  n <- nrow(data)
  k <- length(X)
  ydata <- tbl_dfGrabber(data, y)
  xdata <- tbl_dfGrabber(data, X)
  # beta
  betahat <- solve(t(xdata) %*% xdata) %*% t(xdata) %*% ydata
  # resids
  e <- ydata - xdata %*% betahat
  s2 <- (t(e) %*% e) / (n-k)
  XpXinv <- solve(t(xdata) %*% xdata)
  #std error
  se <- sqrt(s2 * diag(XpXinv))
  # t statistic & its pvalue
  tStat <- (betahat - 0) / se
  pVal <- tStat %>%
    abs() %>%
    "*"(-1) %>%
    pt(df = n - k) %>%
    "*" (2)
  olsOut <- list(betahat, se, tStat, pVal)
  names(olsOut) <- c("betahat", "SE", "tStat", "tStatPVal")
  return(olsOut)
}
```

```
##### RANDOMIZATION SEED
set.seed(12345)
```

```
##### PARALLEL SETUP
cores <- detectCores()
cl <- makeCluster(cores - 1)
clusterEvalQ(cl, library(dplyr))
clusterEvalQ(cl, library(ggplot2))
clusterSetRNGStream(cl, 12345)
```

```
##### GGLOT SETUP
myThemeStuff <- theme(panel.background = element_rect(fill = NA),
  panel.border = element_rect(fill = NA, color = "black"),
  panel.grid.major = element_blank(),
  panel.grid.minor = element_blank(),
  axis.ticks = element_line(color = "gray5"),
  axis.text = element_text(color = "black", size = 10),
  axis.title = element_text(color = "black", size = 12),
  legend.key = element_blank()
)
```

Okay, great. Let's go ahead and set up a function that will calculate our sample size (note that this looks similar, but not identical, to the formula above, because I've rearranged it to be in terms of N rather than MDE):

```
sampSi <- function(alpha, kappa, P, sigma2, MDE) {
  # set the DOF nice and high on the t distr
  tkappa <- pt((1-kappa), 100000)
  talpha <- pt(alpha, 100000)
  firstFrac <- ((tkappa + talpha)^2) / (P * (1-P))
  secondFrac <- sigma2 / (MDE^2)
  myN <- firstFrac * secondFrac
  return(myN)
}
```

Let's figure out what N needs to be in our farmer example. Suppose that we're trying to detect a 1 bushel per acre increase in yields, using $\alpha = 0.05$, $\kappa = 0.80$, and $P = 0.5$. We'll guesstimate that our residual variance is 20.

```
myN <- sampSi(0.05, 0.80, 0.5, 20, 0.25)
myN

## [1] 1546.543
```

Et voila! Sort of. Running an experiment with a fraction of a farmer is tricky. Do we treat everything except your head? Let's update our `sampSi()` function:

```
sampSi <- function(alpha, kappa, P, sigma2, MDE) {
  # set the DOF nice and high on the t distr
  tkappa <- pt(kappa, 100000)
  talpha <- pt(alpha, 100000)
  firstFrac <- ((tkappa + talpha)^2) / (P * (1-P))
  secondFrac <- sigma2 / (MDE^2)
  # round up to the nearest integer
  myN <- ceiling(firstFrac * secondFrac)
  return(myN)
}
```

And test again:

```
myN <- sampSi(0.05, 0.80, 0.5, 20, 0.25)
myN

## [1] 2191
```

Much better. Notice that if you wanted to, you could actually use this to trace out a MDE-sample size curve. The best way to do this is to calculate the necessary sample size for a bunch of different MDEs. We'll turn to our old friend `sapply()` for this:

```
myMDEs <- seq(0.1, 1, 0.01)

mySampSis <- function(alpha, kappa, P, sigma2, MDEvec) {
  out <- sapply(MDEvec, function(x) {
    sampSi(alpha, kappa, P, sigma2, x)
  })
}

sizes <- mySampSis(0.05, 0.80, 0.5, 20, myMDEs)

pcData <- cbind(myMDEs, sizes) %>%
  as.tbl_df()

names(pcData) <- c("MDE", "N")
```

As usual, we'd like to actually look at these results. We could take a look at the first few rows of the data:

```
pcData

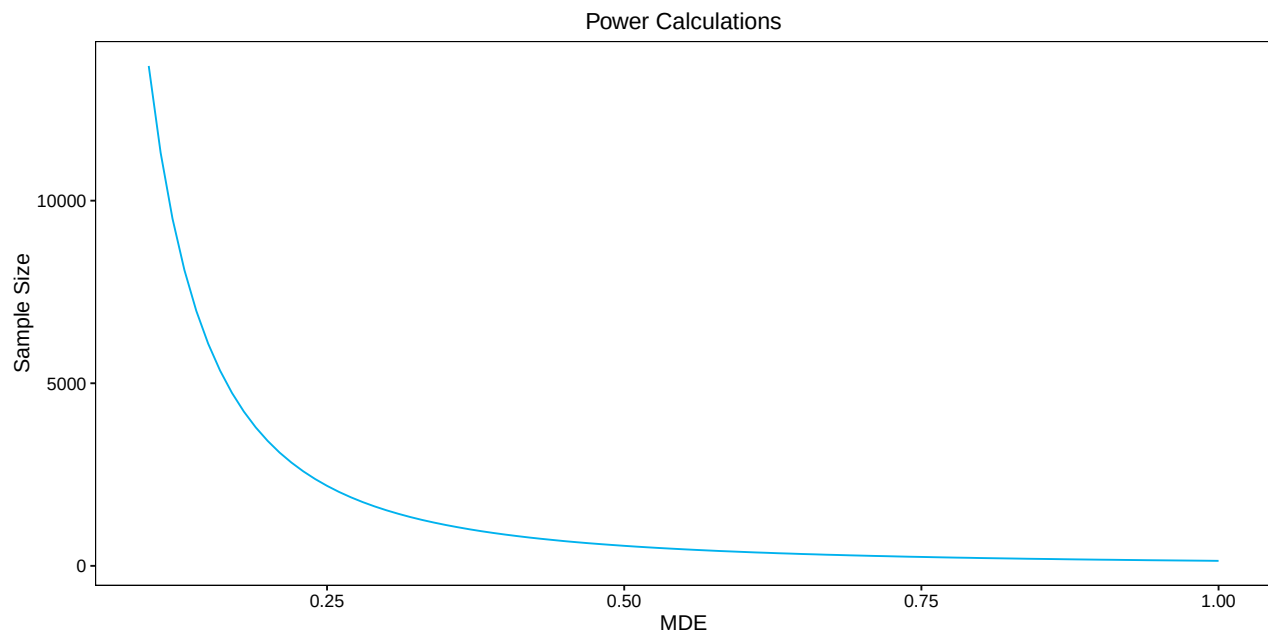
## Source: local data frame [91 x 2]
##
##      MDE      N
##    (dbl) (dbl)
## 1  0.10 13689
## 2  0.11 11313
## 3  0.12  9506
```

```
## 4    0.13  8100
## 5    0.14  6984
## 6    0.15  6084
## 7    0.16  5348
## 8    0.17  4737
## 9    0.18  4225
## 10   0.19  3792
## ..    ...    ...
```

But a more effective way of looking at this is just to plot the results:

```
rctPlot <- ggplot(data = pcData, aes(x = MDE, y = N)) +
  geom_line(color = "deepskyblue2") + myThemeStuff +
  xlab("MDE") + ylab("Sample Size") +
  ggtitle("Power Calculations")
```

```
rctPlot
```



As our MDE increases, the number of observations we need falls, just like we should expect. Cool! By the way: we're once again in kind of uncharted waters here. There isn't really a canned routine to do power calculations well - so consider these section notes a first stab at exactly that. ARE 212 World Premiere (number 2)!

All I really need to know I learned from simulations

It turns out that doing power calculations by using this formula is very constraining. Setting α and κ shouldn't make you uncomfortable, but making a guess at σ^2 should. Furthermore, when you use the formula, you're assuming that you have a perfect RCT design with *iid* errors, which isn't very flexible (you make this assumption

when you plug stuff in for $\sqrt{\mathbb{V}(\hat{\beta})}$.⁹ How can we do better? We can do power calculations *by simulation*.

We'll first do this by playing God and setting up our own complete dataset and DGP, and then we'll extend this by playing around with "real" data. Using simulation to conduct our power calculations will allow us to break free of the formula, and instead focus on finding MDE-sample size pairs in which our chosen research design will yield appropriately-powered results. To do this, recall that when our experiment is appropriately powered, we will correctly reject a false null 80% of the time. We can use this fact to tell whether our experiment is powered or not. We'll proceed in several steps. First, create a set of candidate MDEs and candidate sample sizes. For each MDE-sample size pair:

1. Generate a dataset of your hypothesized sample size, using your hypothesized treatment effect.
2. Run the regression appropriate for your research design.
3. Calculate the t -statistic on your coefficient of interest.
4. Repeat the above steps many times.
5. Save the fraction of t -statistics that are larger than 1.96.

Any MDE-sample size combination for which the fraction of t -statistics that are larger than 1.96 is greater than or equal to 80% is "well-powered." That sounds complicated, but it's not actually that bad. Let's go ahead and code it up. Let's think about our Kenyan farmers: we'll test treatment effects of between 0.1 and 0.5 bushels / acre, using a step size of 0.025, and we'll allow for possible sample sizes of 10 to 1000, using a step size of 50. How did I pick these numbers? Kind of arbitrarily and with lots of trial and error (you're welcome). Did I mention yet that power calculations are just as much art as they are science?¹⁰ Be aware that these functions can take a little while to run - and the more MDEs and sample sizes you pick, the longer they'll take.

```
guessNs <- seq(10, 1000, 50)
guessMDEs <- seq(0.1, 0.5, 0.025)
```

For each MDE-sample size pair, we need to generate some data, run our regression of interest, and save our t -statistic. The first thing we'll need to do is to write a function to randomly assign treatment to some proportion of our population. We could do this in one of two ways: we could flip a (potentially weighted) coin for each person, or we could run a random number generator and treat the observations in the largest p percent of rows. The second method is better, because it guarantees you will treat the correct percent of people, so we'll do this:

```
randomizer <- function(data, p) {
  # randomly sample p % of the rows in your dataset
  randomObs <- 1:nrow(data) %>%
    sample(size = nrow(data) * p)
  # create a treatment variable = 1 if your row was sampled
  data <- mutate(data,
    t = ifelse(row_number() %in% randomObs, 1, 0))
  return(data$t)
}
```

⁹As an example: everybody *loves* to cluster their standard errors these days. But if you do power calculations without explicitly taking into account clustering, and then cluster your standard errors after running your experiment, your power calculation isn't going to match your actual power level. Trust me on this one.

¹⁰We're actually trying to estimate a continuous surface here - this is kind of similar to when we did MLE and searched over points using the `optim()` function.

Now that we have this, we can create a function that simulates our experiment. Notice that there's no free lunch here: we still technically have to choose the mean of our dependent variable (this is what we're doing when we set the intercept to 50 in our DGP), and the variance of our error term (this is like picking σ^2 above). In this toy example, we're drawing errors from a Normal distribution with mean of 0 and SD of 1 - but be aware that if you're trying to do this for an experiment you're interested in running, you'll want to adjust these parameters to better reflect your empirical context.¹¹

```
myDraw <- function(n, mde) {
  #random noise as usual
  e <- rnorm(n, mean = 0, sd = 1) %>%
    as.tbl_df()
  # treatment vector! we're hard-coding p = 0.5
  t <- randomizer(e, 0.5)
  # set up your DGP. Notice that this depends on MDE
  y <- 50 + mde * t + e
  # create a dataset
  myData <- cbind(y, 1, t) %>%
    as.data.frame()
  names(myData) <- c("y", "ones", "treat")
  # run your regression
  reg <- OLS(myData, "y", c("ones", "treat"))
  # save the tStat on the treatment variable
  tStat <- reg$tStat[2]
  # save a 1 if the t stat > 1.96, and 0 otherwise
  success <- ifelse(tStat > 1.96, 1, 0)
  return(success)
}
```

Let's test this with our results from before:

```
testDraw <- myDraw(137, 0.25)
testDraw

## [1] 0
```

Success! Now we want to be able to do this *a bunch* of times with *a bunch* of possible MDEs and *a bunch* of possible sample sizes. No problem: we just need *a bunch* of `apply()`s. The first thing we want to do is run our `myDraw()` function around 250 times for a given β and sample size. In real life you'll want more draws per cell, but in the interest of time in section and the health of my poor little laptop, we'll cap it at 250 for now. Let's wrap `myDraw()` in a function that will do this:

```
myRejects <- function(n, mde) {
  # run myDraw 250 times
  out <- sapply(1:250, function(x) {
    myDraw(n, mde)
  })
}
```

¹¹Be warned: if you don't do this, you could accidentally end up thinking that you're much more powered than you actually are! This is another advantage of using actual data if you have access to it - you don't have to worry about picking these parameters.

```

})
# take the mean successes in a cell
# remove NAs from this on the off chance they're there
myMean <- mean(out, na.rm = TRUE)
# make our output a 1 x 3 dataframe
output <- cbind(n, mde, myMean) %>%
  as.tbl_df()
names(output) <- c("N", "MDE", "RejectFrac")
return(output)
}

```

And test this out:

```

testRejects <- myRejects(137, 0.25)
testRejects

## Source: local data frame [1 x 3]
##
##      N    MDE RejectFrac
##   (dbl) (dbl)      (dbl)
## 1   137  0.25      0.296

```

We (correctly) reject the null 29.6% of the time. This test is significantly underpowered: we should reject 80% of the time! Let's now get our function to take a vector of MDEs and a vector of sample sizes as inputs. We want the output of this process to be a giant dataframe where the first column is candidate sample sizes, the second column is candidate MDEs, and the final column is the fraction of draws in a cell in which we reject the null:

```

# first, wrap everything in a function
myPower <- function(samplesizes, MDEs){
  # outer loop over N's
  # use lapply so we get the right output
  # sapply will try to make a matrix
  powers <- lapply(samplesizes, function(y) {
    # inner loop over MDEs
    lapply(MDEs, function(x) {
      myRejects(y, x)
      # this extra step appends each 1x3 df together
    }) %>%
      bind_rows()
    # and bind again for the outer loop
  }) %>%
    bind_rows()
  # output our dataframe
  return(powers)
}

```

We could test it like this:

```
testPower <- myPower(guessNs, guessMDEs)
```

But before you do that, notice that we're drawing a *ton* of samples. This is going to crawl. Let's use our parallel processing skills to make this better on ourselves (it'll still be slow, but not nearly as bad). We've done most of the heavy lifting already; all we need to do now is to send our MDE and sample size guess vectors, as well as our new functions, to the clusters.

```
clusterExport(cl, "as.tbl_df")
clusterExport(cl, "tbl_dfGrabber")
clusterExport(cl, "OLS")
clusterExport(cl, "randomizer")
clusterExport(cl, "myDraw")
clusterExport(cl, "myRejects")
clusterExport(cl, "guessNs")
clusterExport(cl, "guessMDEs")
```

Finally, we'll modify our `myPower()` function to run in parallel:

```
myPower <- function(samplesizes, MDEs){
  powers <- parLapply(cl, samplesizes, function(y) {
    lapply(MDEs, function(x) {
      myRejects(y, x)
    }) %>%
      bind_rows()
  }) %>%
    bind_rows()
  return(powers)
}
```

And take it for a spin:

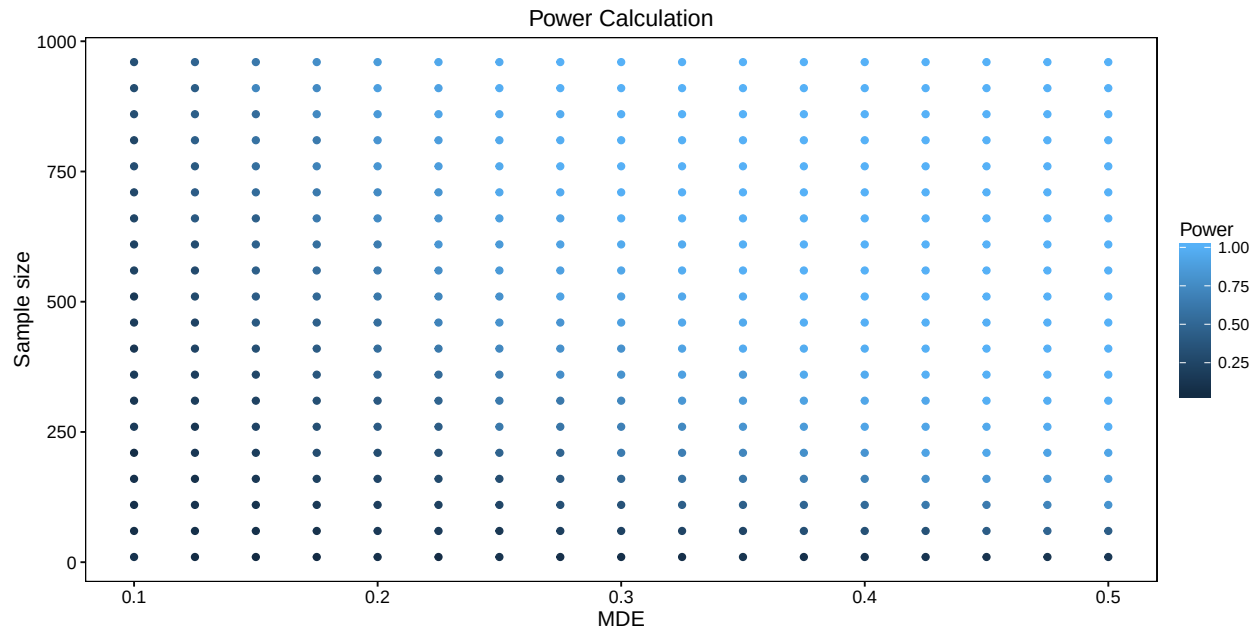
```
powerOut <- myPower(guessNs, guessMDEs)
```

Now we'd like to do something useful with this. The first thing we'd like to do is to plot our data. We could just make a scatter plot of sample sizes and MDEs, colored by their power value, like this:

```
testPlot <- ggplot(data = powerOut,
  aes(x = MDE, y = N, color = RejectFrac)) +
  geom_point() +
  ggtitle("Power Calculation") + xlab("MDE") + ylab("Sample size") +
  labs(color = "Power") +
  myThemeStuff
```

But that's both ugly and hard to read. Remember the nice line we had above? Let's replicate that - except instead of having only one line, now we'll have a bunch, since that was for exactly 80% power; now we have a range of power values in our data. We're going to plot one "contour" for each power decile (10%, 20%, etc), where the region above the 80% contour is powered at traditional levels. To do this, we need to add a couple of columns to our data.

testPlot



First, we want to create a variable that indicates whether each individual point in our MDE-sample size grid represents a well-powered experiment or not.

In order to plot these “contour lines”, we want to grab the lowest N associated with each MDE value for each “power slice” of our data. What does that mean? We’d like to grab all of the points that are powered close to, for example, 10%, and then plot the lower envelope of those points. All points above this line will be powered to at least 10%. Because our data are kind of sparse, we’re not necessarily guaranteed to observe each power decile with each possible effect size. Instead, we’re going to find all of the points that lie within 0.04 of a power decile, and use the MDE-sample size relationship of these points to make our contour lines.¹² This sounds more confusing than it actually is:

```
# create our powered indicator
powerOut <- mutate(powerOut, powered = as.factor(ifelse(RejectFrac >= 0.8, 1, 0)))
# create a variable defining which decile each point is close to
# if any
powerOut <- mutate(powerOut, rounded = round(RejectFrac, digits = 1),
  close = ifelse(abs(rounded - RejectFrac) < 0.04,
    rounded, NA),
  close = as.factor(close)) %>%
  # remove the rounded variable from our dataset
  select(-rounded)
# use group_by to grab each MDE-decile pair
byMDE <- group_by(powerOut, MDE, close) %>%
  # sort by N
  arrange(N) %>%
  #grab the first observation (smallest N)
```

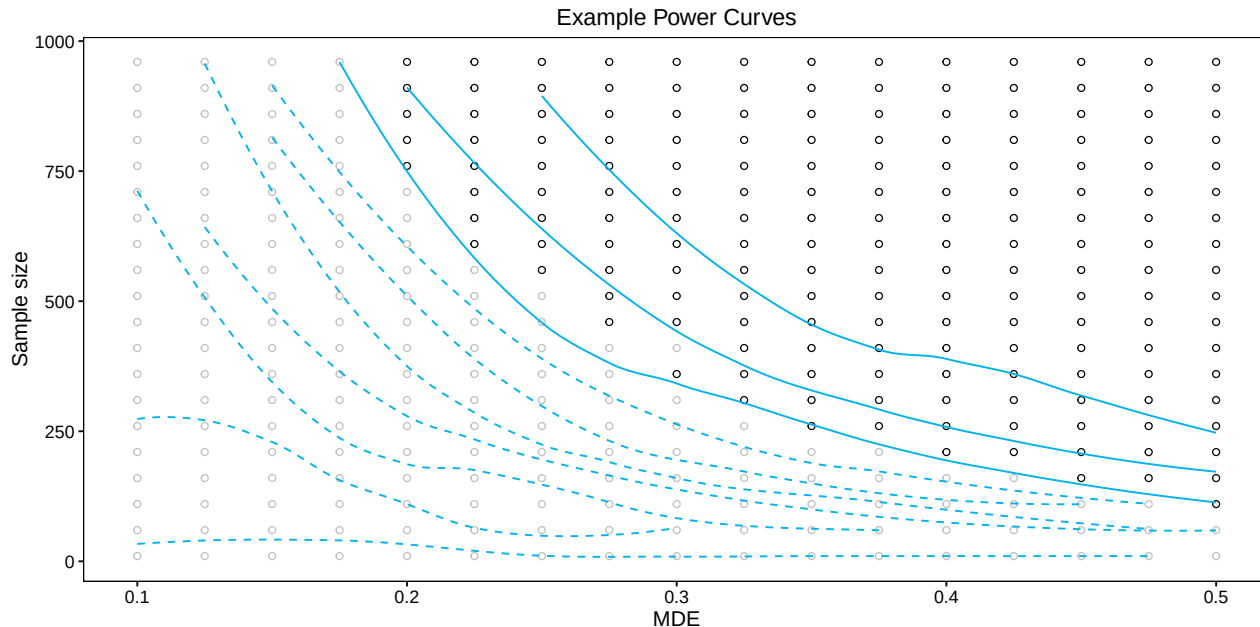
¹²All this is largely because we’re doing this using a small number of draws per cell. The more draws per cell you do, and the more cells you do simulations for, the smaller your bandwidth will have to be to get a nice looking figure here.

```
slice(1) %>%
#undo the grouping
ungroup() %>%
#remove NAs
filter(is.na(close) == FALSE)
```

Now we can plot this thing much more nicely:

```
powerPlot <- ggplot(data = powerOut,
                    aes(x = MDE, y = N)) +
  geom_point(shape = 21, aes(color = powered), show.legend = FALSE) +
  geom_smooth(data = byMDE, se = FALSE,
             size = 0.5, color = "deepskyblue2",
             aes(linetype = close), show.legend = FALSE) +
  scale_color_manual(values = c("gray75", "black")) +
  scale_linetype_manual(values = c(rep("dashed", 7),
                                   rep("solid", 3))) +
  ggtitle("Example Power Curves") +
  xlab("MDE") + ylab("Sample size") +
  myThemeStuff
```

powerPlot



Each gray dot represents an under-powered MDE-sample size combination; each black dot is adequately powered. Each line is one of our “contours”, with dashed lines indicating power of 10%, 20%, ..., 70%, and solid lines indicating power of 80%, 90% or 100%. Whoo! Note that these lines will be nicer and nicer the more data that you add: more draws per cell and more cells will give you a more accurate power picture.

[Power calculations] are a girl's best friend

The really nice thing about this setup is that we can easily adapt it to other situations. In particular, the ideal way of doing power calculations is to get your hands on a real dataset with your outcomes of interest from the setting where you're planning to do your experiment (a pilot can help with this), and then simulate treatment effects into this data. Why is this the ideal? This will allow you to capture underlying patterns in the data without having to try to construct them yourself.

In this type of power calculation, the steps are very similar to what we did above, except rather than building up a dataset from scratch, we'll sample observations from our original dataset, and then add in a treatment effect. We'll try this out with the `diamonds` dataset from `ggplot2`. Let's say we've come up with a new amazing diamond shaping procedure. We're interested in the effect this has on prices. We'll simulate treatment effects of between 100 and 500 dollars, and sample sizes of between 1,000 and 10,000 diamonds.

```
dataNs <- seq(50, 1000, 100)
dataMDEs <- seq(500, 2000, 50)
```

This time, we'll set up our draw function slightly differently, such that we start by drawing a sample from our dataset of interest. We'll then "treat" 50% of this sample, and apply our treatment effect by changing the value of our pre-existing outcome variable. Finally, run the regression and save whether we rejected or not as before.

```
dataDraw <- function(n, mde, data) {
  # first randomly sample from data
  myData <- sample_n(data, n)
  t <- randomizer(myData, 0.5)
  ## THIS TIME, ADD TREATMENT TO THE EXISTING Y (WE'VE HARD-CODED PRICE IN HERE)
  myData <- mutate(myData, treatment = t, ones = 1,
                    y = price + mde*treatment)
  # run your regression
  reg <- OLS(myData, "y", c("ones", "treatment"))
  # save the tStat on the treatment variable
  tStat <- reg$tStat[2]
  # save a 1 if the t stat > 1.96, and 0 otherwise
  success <- ifelse(tStat > 1.96, 1, 0)
  return(success)
}
```

Everything else is the same:

```
dataRejects <- function(n, mde, data) {
  out <- sapply(1:100, function(x) {
    dataDraw(n, mde, data)
  })
  myMean <- mean(out, na.rm = TRUE)
  output <- cbind(n, mde, myMean) %>%
    as.tbl_df()
  names(output) <- c("N", "MDE", "RejectFrac")
}
```

```

    return(output)
  }

dataPower <- function(samplesizes, MDEs, data){
  powers <- parLapply(cl, samplesizes, function(y) {
    lapply(MDEs, function(x) {
      dataRejects(y, x, data)
    }) %>%
      bind_rows()
  }) %>%
    bind_rows()
  return(powers)
}

```

And export stuff to the clusters:

```

clusterExport(cl, "dataDraw")
clusterExport(cl, "dataRejects")
clusterExport(cl, "dataNs")
clusterExport(cl, "dataMDEs")

```

And finally, run it:

```
myDataPower <- dataPower(dataNs, dataMDEs, diamonds)
```

Just like before, we can make a nice power plot:

```

dataPowerOut <- mutate(myDataPower, powered = as.factor(ifelse(RejectFrac >= 0.8, 1, 0)))
dataPowerOut <- mutate(dataPowerOut, rounded = round(RejectFrac, digits = 1),
  close = ifelse(abs(rounded - RejectFrac) < 0.04,
    rounded, NA),
  close = as.factor(close)) %>%
  select(-rounded)

dataByMDE <- group_by(dataPowerOut, MDE, close) %>%
  arrange(N) %>%
  slice(1) %>%
  ungroup() %>%
  filter(is.na(close) == FALSE)

```

```

dataPowerPlot <- ggplot(data = dataPowerOut,
  aes(x = MDE, y = N)) +
  geom_point(shape = 21, aes(color = powered), show.legend = FALSE) +
  geom_smooth(data = dataByMDE, se = FALSE,
    size = 0.5, color = "deepskyblue2",

```

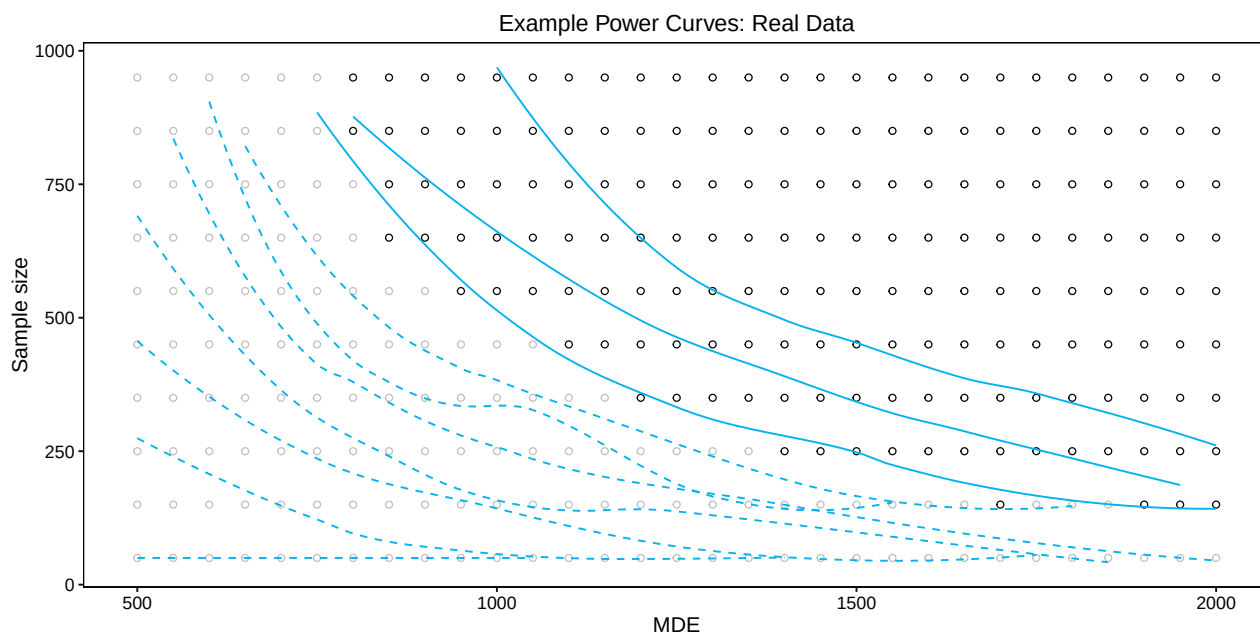
```

aes(linetype = close), show.legend = FALSE) +
scale_color_manual(values = c("gray75", "black")) +
scale_linetype_manual(values = c(rep("dashed", 7),
                                rep("solid", 3))) +
ggtitle("Example Power Curves: Real Data") +
xlab("MDE") + ylab("Sample size") +
myThemeStuff

```

Et voila: Whoo! We are officially power calculation masters.

dataPowerPlot



It's important to note that while we've just coded up two sets of power calculations for pretty simple situations, you can easily extend this general framework of choosing a sample size and a possible MDE, setting up a DGP that matches your particular setting, simulating a bunch of runs of sample size - MDE combinations, and saving the fraction of correct rejections of the null to a variety of settings. Rather than using a simple RCT framework, for example, you could extend this to difference-in-differences, IV, regression discontinuity, etc just by changing the DGP you're simulating. Simulation-based power calculations are very powerful, and as we've seen, pretty easy to implement.¹³ We didn't really touch on it here, but they're particularly useful in settings where there are patterns in your underlying data that aren't super straightforward to model - for example, incorporating clustered standard errors into simulation-based power calculations is easy; doing this in formula-based power calculations is super hard.¹⁴

We'll stop here and pick up next time with some introductory spatial data analysis.

¹³Matt, Louis, and I are working on putting together some canned routines that will even do most of the heavy lifting for you! Check back with us in fall 2016.

¹⁴The equation to do this correctly takes up about a full page of typeset math - not to mention that using it requires you to make a *ton* of assumptions about parameters.